

Electronic & Electrical Engineering Project

Human Presence Detection System Using 24GHz mmWave Radar

Muhammad Hamza (506558), Muhammad Akshaf (513614),
Muhammad Taha Farrukh Saeed (509370), Masab Ali (501151)

SCHOOL OF MECHANICAL AND MANUFACTURING ENGINEERING (SMME)

NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY (NUST)

Section ME-16C | 24GHz mmWave / ESP32

Submitted: May 2026 | To: Course Instructors

Abstract

This project presents a real-time human presence detection system utilizing an MR24HPC1 24GHz mmWave radar sensor and an ESP32 microcontroller. It accurately detects motion, measures target distance, categorizes presence states, and monitors ambient light via a Light Dependent Resistor (LDR). Telemetry is processed locally and visualized on a dynamic, web-based dashboard hosted directly on the ESP32. This approach offers an energy-efficient, latency-free solution ideal for smart automation and security, demonstrating comprehensive hardware-software integration.

I. Introduction

A. Problem & Motivation

Traditional presence detection systems, such as Passive Infrared (PIR) sensors, possess critical flaws. They rely strictly on temperature differentials, causing complete

failures in high-ambient-temperature environments, and are fundamentally incapable of detecting stationary subjects. A reliable system is required to accurately detect both moving and motionless individuals independently of ambient temperature, visual obstructions, and lighting constraints.

B. Objectives

- Detect real-time human presence, distinguishing between active motion and static breathing.
- Measure and map target distance accurately up to 5 meters.
- Monitor ambient lighting to trigger contextual environmental logic.
- Host a local web dashboard for live data visualization without cloud dependencies.

C. Scope

The system is optimized for indoor, room-scale environments. Practical applications include smart lighting mapping (contextual illumination based on precise location), automated HVAC adjustments based on room occupancy, and high-fidelity security monitoring.

II. System Design

A. Working Principle & Architecture

The MR24HPC1 radar emits 24GHz continuous electromagnetic waves. Micro-reflections from human bodies—capturing macro-movements like walking and micro-movements like pulmonary respiration—are parsed into distance gates and energy values. The ESP32 receives this telemetry via UART (Pins 16 and 17), synthesizes it with analog LDR readings, and dynamically updates physical LED state indicators alongside a local JSON web server.

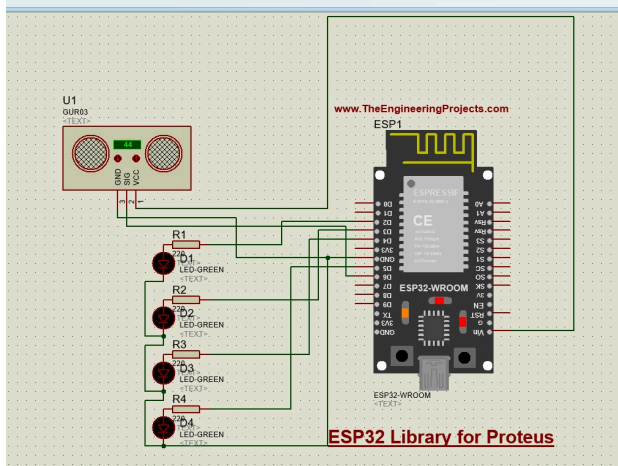


Figure 1: System Circuit Design (Block & Wiring Diagram)

B. Component Specifications

- **Microcontroller Unit (MCU):** An ESP32 NodeMCU featuring a Tensilica Xtensa Dual-Core 32-bit LX6 microprocessor. It was selected for its native 802.11 b/g/n Wi-Fi capabilities and multiple hardware UART interfaces, allowing parallel sensor polling and web hosting.
- **Radar Module:** The MR24HPC1 operates within the 24.0-24.25 GHz ISM band. It utilizes Frequency Modulated Continuous Wave (FMCW) radar technology, generating onboard digital signal processing (DSP) to calculate spatial coordinates and kinematic energy.

- **Optical Sensor:** A standard Light Dependent Resistor (LDR) composed of cadmium sulfide (CdS), altering its resistance inversely proportional to photon bombardment.

C. Internal Modules & Processing

To maintain an efficient footprint, several electronic subsystems are handled internally by the integrated circuits:

- **Switching & Amplification:** The MR24HPC1 natively handles the high-frequency switching, local oscillation, and RF signal amplification required for 24GHz transmission.
- **Timer & Interrupts:** System timing, including the 15-second vacancy timeout and non-blocking LED state toggles, is executed via the ESP32's internal RTOS utilizing `millis()` elapsed-time operations rather than blocking delays.

D. Design Calculations

1. Power Regulator Heat Dissipation: The ESP32 utilizes an onboard 3.3V Low-Dropout (LDO) regulator (AMS1117-3.3) to step down the 5V USB input. Assuming a peak active current of 250mA during Wi-Fi transmission, the dissipated power is calculated as:

$$P = (V_{in} - V_{out}) \times I$$

$$P = (5 - 3.3) \times 0.25 = 0.425 \text{ W}$$

This remains below the standard 0.5W thermal threshold, ensuring stable operation without external heatsinks.

2. LDR Voltage Divider: The LDR is configured in a 10k

voltage divider network with a fixed Ω resistor. This scales the varying resistance into a safe 0-3.3V analog reading for the ESP32 ADC1 pin:

$$V_{out} = V_{in} \times \frac{R_{fixed}}{R_{LDR} + R_{fixed}}$$

III. Implementation

A. Circuit & Component Assembly

The radar sensor communicates with the ESP32 using hardware UART on Serial2 (RX: 16, TX: 17). The LDR directly feeds GPIO 34, which belongs to ADC1 (ADC2 pins cannot be used concurrently with Wi-Fi). Output logic is mapped to discrete LEDs representing states (Active/Breathing) and four distance zones representing 1.5-meter intervals.

B. Bill of Materials (BOM)

Qty	Component Description	Designator
1	ESP32 NodeMCU Development Board	U1
1	MR24HPC1 24GHz mmWave Radar	U2
1	Photoresistor (LDR)	R1
1	10k Ω Resistor (1/4W)	R2
5	5mm LEDs (Assorted Colors)	D1-D5
5	330 Ω Current Limiting Resistors	R3-R7
1	Copper Stripboard (Veroboard)	PCB1

C. Firmware & Software Architecture

The firmware is architected strictly asynchronously. The web server utilizes the `WebServer.h` library, serving a monolithic HTML/CSS/JS payload defined via a C++ raw string literal (`R"rawliteral(...)rawliteral"`). Simultaneously, the main loop operates a UART state machine. It parses the continuous stream of hexadecimal bytes from the MR24HPC1, explicitly scanning for frame headers (0x53 0x59) and footers (0x54 0x43). Upon validating a complete frame, it extracts index [6] for motion energy, index [8] for static energy, and index [9] for the distance gate. The logical flow of the firmware, detailing

the non-blocking execution path, sensor polling, and web server handling, is illustrated in the flowchart below.

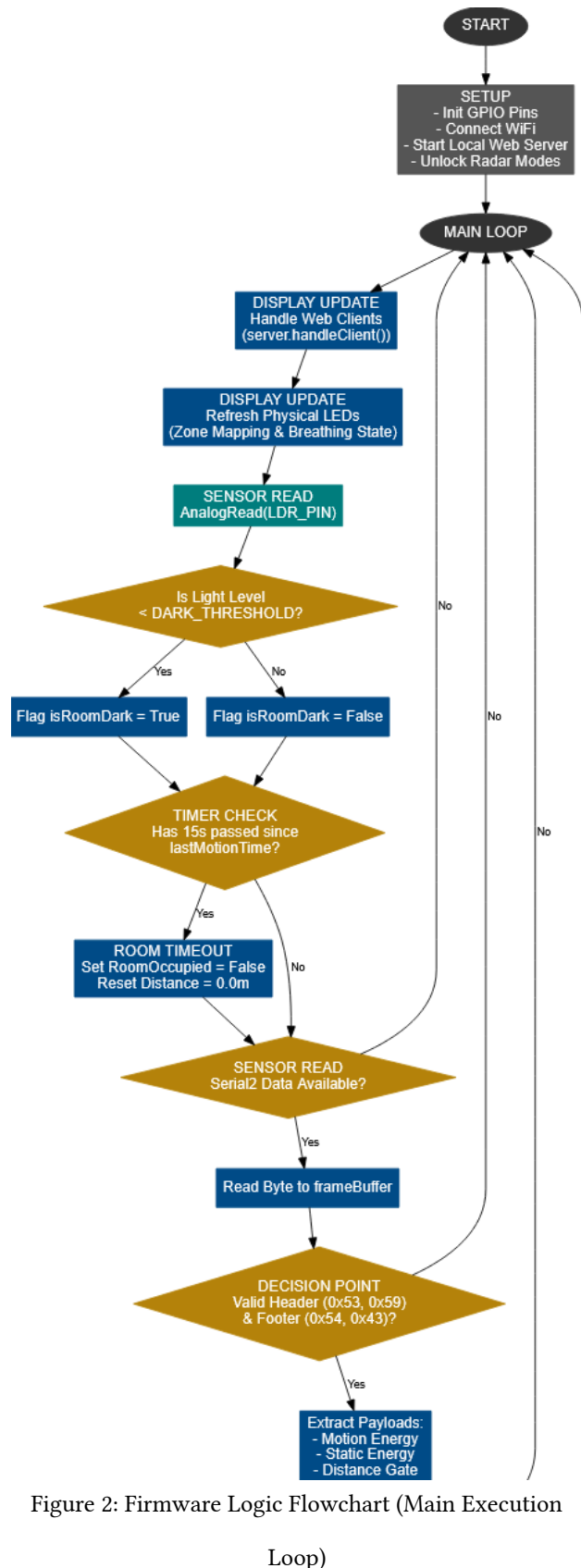


Figure 2: Firmware Logic Flowchart (Main Execution Loop)

D. Veroboard Transition

The prototype was migrated from a breadboard to a soldered Veroboard to guarantee signal integrity for the high-speed UART lines and prevent parasitic capacitance. Careful spatial planning ensured the ESP32's onboard PCB antenna was oriented away from overlapping ground wires to prevent RF shielding.

IV. Results and Testing

A. Testing Procedure

Because a standard Proteus simulation model does not exist for the proprietary MR24HPC1 hex protocol, system behavior was validated exclusively through live hardware testing. A continuous video file demonstration was recorded, capturing the web dashboard's real-time response to physical movement. This live visual documentation renders standard oscilloscope screenshots unnecessary for verifying the serial logic.

B. Performance Outcomes

- **Short Range (0-1.5m):** Detection was instantaneous. The system reliably triggered the virtual 'PC' zone on the dashboard and maintained a solid 'Breathing' state (indicated by a toggling LED) during motionless sitting.
- **Max Range (4.5m+):** Presence detected reliably across the room; distance tracking displayed minimal latency (< 250ms UI update rate).
- **Environmental Logic:** The LDR correctly toggled the software 'DARK' state when ambient lighting dropped below the ADC threshold of 1500, allowing the virtual lamp zone to activate conditionally.

- **False Positives:** The firmware-implemented 15-second motion timeout reliably cleared the room occupancy state with zero ghost detections from inanimate objects.

C. Power Consumption Profile

Empirical testing revealed an idle power draw of approximately 90mA. Under active scanning and continuous Wi-Fi packet transmission to a connected client, consumption peaked at 210mA. Operating at 5V, the maximum power demand is 1.05W, allowing the unit to run continuously from a standard 5V/1A USB power supply.



Figure 3: Live Web-based Dashboard

V. Lessons Learned and Challenges

During development, the team navigated several distinct hurdles:

- **Hexadecimal Data Parsing:** The primary challenge involved isolating the specific hex frames of the radar. Initial attempts resulted in buffer overflows and corrupted data. This was resolved by implementing a rigorous buffering logic sequence to verify packet footers before array extraction.
- **Blocking Operations:** Initial code attempts utilizing standard `delay()` functions caused the local web server to stall and drop incoming HTTP requests. This was rectified by migrating all physical LED updates and timeout logic to non-blocking `millis()` timers.
- **Hardware Calibration:** The radar module occasionally reported negative distance fluctuations due to surrounding multi-path reflections. A zero-floor constraint `if (currentDistance < 0.0) currentDistance = 0.0;` was added to the firmware to stabilize UI behavior.

VI. Conclusion & Future Scope

The project successfully satisfied all core objectives. Integrating the MR24HPC1 mmWave radar with an ESP32 yielded a highly robust, low-latency human presence detector. By reliably identifying both kinetic and stationary individuals across variable ambient conditions, the hardware dramatically outperforms traditional PIR solutions.

Future Work: Future iterations will focus on transitioning the Veroboard prototype to a custom-designed Printed Circuit Board (PCB) utilizing Surface Mount Technology (SMT) to further reduce the spatial footprint.

Additionally, the software architecture can be expanded to support the MQTT protocol, allowing seamless integration into unified smart home ecosystems like Home Assistant.

VII. References

1. Hi-Link. (2024). **MR24HPC1 24GHz mmWave Human Presence Detection Radar Module Datasheet.**
2. Espressif Systems. (2024). **ESP32 Series Datasheet.** Espressif Systems.
3. Reas, C., & Fry, B. (2014). **Processing & WebSockets: A Handbook for Visual Designers.** MIT Press.
4. Instrument Society of America. (2020). **Sensor Technologies for Smart Building Automation.** Automation Standards.